



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO DO UÍGE
CURSO DE ENGENHARIA INFORMÁTICA

TRABALHO EM GRUPO DE S.G.B.D II

Catálogo de Produtos Dinâmico

3º ano
Turma:
TL 101
Período
:
Manhã

ORIENTADOR

Moyo Kanivengidio, Msc.

Uíge, Maio de 2026



UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO DO UÍGE
CURSO DE ENGENHARIA INFORMÁTICA

Catálogo de Produtos Dinâmico para E-commerce de Alta Escala

1. Filipe Pedro Muanza (Nº Mec.: 17684)
2. Filipe Miguel João (Nº Mec.:)
3. Guiola André (Nº Mec.: 17690)
4. Nsimba Alberto Samuel (Nº Mec.: 17736)
5. Pedro Marcos Kudikussa (Nº Mec.:)

3º Ano de Engenharia Informática

Uíge, Maio de 2026

INDICE

0. INTRODUÇÃO	1
2. ANÁLISE DO DOMÍNIO E PADRÕES DE ACESSO.....	3
4. MODELAÇÃO DE DADOS	5
5. IMPLEMENTAÇÃO E RESULTADOS	7
5.1. Ambiente de Testes.....	7
5.2. Resultados Reais das Consultas	7
6. DISCUSSÃO CRÍTICA	10
6.3 Tolerância a Falhas e Partições de Rede	11
6.4. Análise de Desempenho Conclusões	11
CONCLUSÃO	13

0. INTRODUÇÃO

O presente trabalho foi desenvolvido no âmbito da disciplina de Sistemas de Gestão de Bases de Dados II, sob orientação do docente Moyo Kanivengidio, e tem como objectivo a concepção, implementação e avaliação de uma camada de persistência NoSQL para um catálogo de produtos de e-commerce de alta escala, utilizando o MongoDB 7.0 em configuração de Replica Set com três nós.

O subdomínio escolhido Catálogo de Produtos Dinâmico representa um dos componentes mais críticos de qualquer plataforma de e-commerce moderna, exigindo gestão eficiente de milhões de produtos com atributos altamente variáveis, hierarquias de categorias complexas e suporte a pesquisa facetada em tempo real com latências abaixo de 200 ms.

O grupo assumiu este projecto como uma oportunidade concreta de consolidar os conhecimentos adquiridos em sala de aula sobre bases de dados NoSQL, modelação orientada a consultas (Query-Driven Design), estratégias de indexação, Aggregation Pipeline e arquitecturas distribuídas com alta disponibilidade. Cada decisão de design foi tomada com base nos princípios teóricos do Teorema CAP, nos padrões de acesso identificados e nas métricas reais obtidas durante a implementação.

O sistema foi implementado sobre 309.469 documentos reais 100.000 produtos, 9.469 utilizadores e 200.000 avaliações, tendo sido criados 12 índices e executadas 7 consultas avançadas com Aggregation Pipeline, cujos resultados de desempenho são documentados e analisados criticamente neste relatório.

1. RESUMO EXECUTIVO

O presente trabalho aborda a concepção e implementação de uma camada de persistência NoSQL para o subdomínio de Catálogo de Produtos Dinâmico de um ecossistema de e-commerce de alta escala. O principal desafio é a necessidade de gerir milhões de produtos com atributos altamente variáveis, hierarquias de categorias complexas e suportar pesquisa facetada em tempo real com latências abaixo de 200 ms.

Foi seleccionado o MongoDB 7.0 como SGBD principal, fundamentado no posicionamento CP do Teorema CAP, na flexibilidade do modelo de documentos JSON e na maturidade dos seus mecanismos de indexação e Aggregation Pipeline. O sistema foi implementado em cluster de três nós (Replica Set rs0) com Docker para garantir alta disponibilidade e reprodutibilidade.

1.1. Métricas Reais da Implementação

- 309.469 documentos inseridos: 100.000 produtos 9.469 utilizadores 200.000 avaliações
- 12 índices criados: compostos, full-text (ponderado), geoespacial 2dsphere e TTL
- 7 consultas avançadas com Aggregation Pipeline: \$facet, \$lookup, \$geoWithin, \$unwind, \$text
- Latência mais rápida: 91 ms (Q3 Full-Text) | Mais lenta: 81 s (Q7 \$lookup sem índice directo)
- Cluster de 3 nós: mongo1 PRIMARY (priority=2) + mongo2 SECONDARY + mongo3 SECONDARY
- Hardware utilizado: PC Windows, Docker Desktop, Python 3.14, pymongo 4.17

2. ANÁLISE DO DOMÍNIO E PADRÕES DE ACESSO

O subdomínio escolhido Catálogo de Produtos Dinâmico representa um dos componentes mais críticos de qualquer plataforma de e-commerce. As suas características fundamentais são:

- Esquema heterogéneo: um Smartphone tem atributos RAM, SO, câmara; um Livro tem ISBN, autor, páginas. Um RDBMS exigiria dezenas de tabelas ou colunas nulas o MongoDB resolve com schema-less.
- Alta frequência de leitura: ratio típico 95% leituras vs 5% escritas. O catálogo é lido em cada page view mas raramente actualizado.
- Pesquisa facetada: filtros simultâneos por categoria, preço, marca, rating e disponibilidade de stock exige índices compostos eficientes.
- Escalabilidade horizontal: crescimento previsível para 10M+ produtos com sharding por categoria.

2.1. Padrões de Acesso (Query-Driven Design)

ID	Padrão de Acesso	Frequência	Tipo
PA-01	Listagem por categoria + filtros preço/marca/rating	Muito Alta	Leitura
PA-02	Pesquisa full-text por nome/descrição/marca	Muito Alta	Leitura
PA-03	Detalhe do produto (atributos + avaliações resumidas)	Alta	Leitura
PA-04	Produtos em destaque / tendência	Alta	Leitura
PA-05	Actualização de stock após venda	Alta	Escrita
PA-06	Adicionar/editar produto	Média	Escrita
PA-07	Análise de receita por categoria (BI)	Baixa	Leitura
PA-08	Produtos disponíveis por proximidade geográfica	Média	Leitura
PA-09	Histórico de preços de um produto	Baixa	Leitura
PA-10	Produtos com avaliações recentes positivas	Média	Leitura

3. JUSTIFICAÇÃO TECNOLÓGICA

3.1. Análise Comparativa

Tecnologia	Vantagens	Limitações	Adequação
MongoDB (Documentos)	Schema flexível JSON/BSON Aggregation Pipeline Índices compostos e full-text Replica Set nativo GeoJSON nativo	JOIN limitado entre colecções Não ideal para relações densas	5%
Apache Cassandra (Família Colunas)	Escalabilidade linear AP Alta disponibilidade Escrita muito rápida	Query-first muito rígido Sem full-text nativo Sem pesquisa facetada	2%
Redis (Chave-Valor)	Latência sub-milissegundo Ótimo para cache/sessões	Dados em RAM (custo elevado) Sem aggregation complexa Não adequado a catálogos	2%

3.2. Teorema CAP e PACELC

O Teorema CAP afirma que um sistema distribuído apenas pode garantir simultaneamente dois de três atributos: Consistência (C), Disponibilidade (A) e Tolerância a Partições (P). Para um catálogo de produtos, a Consistência é crítica um utilizador não deve ver um preço desactualizado após uma promoção. Por isso o MongoDB foi escolhido como sistema CP.

Propriedade	MongoDB (RS)	Cassandra	Redis
Consistência (C)	Forte — leituras no PRIMARY	Eventual (tunable)	Forte (modo single)
Disponibilidade (A)	Alta — failover ~10s	Muito Alta	Alta
Tolerância Partições	Sim	Sim	Sim
Classificação CAP	CP	AP	CP/AP
Latência escrita	Baixa-Média	Muito Baixa	Muito Baixa

4. MODELAÇÃO DE DADOS

A modelação segue o princípio Query-Driven Design: o esquema é otimizado para os padrões de acesso identificados, em detrimento de normalização formal.

4.1. Estratégias de Desnormalização

Estratégia	Justificação
Embedding ratings_summary	Evita JOIN com colecção reviews para PA-01/02/03. Inclui média, total e distribuição directamente no produto.
Embedding price_history	Array de histórico dentro do documento. Permite PA-09 sem query adicional. Limitado a ~12 entradas.
Embedding warehouse_stock	Stock por armazém com coordenadas GeoJSON. Habilita PA-08 (\$geoWithin). Evita JOIN com colecção de armazéns.
Embedding skus	Variantes do produto embutidas como array. Actualização atómica de stock via \$inc.
Referenciação de reviews	Avaliações completas em colecção separada (PA-10). Evita documentos enormes. Usa \$lookup quando necessário.
Campo in_stock calculado	Booleano desnormalizado derivado de total_stock > 0. Permite filtro rápido em PA-01 sem calcular soma de arrays.

4.2. Estrutura do Documento Produto

Abaixo apresenta-se a estrutura JSON do documento produto, que incorpora todas as estratégias de desnormalização descritas:

```
{
  "product_id": "PROD-000001",
  "name": "Samsung Smartphone Galaxy X 4521",
  "brand": "Samsung",
  "category": "Eletrónicos",
  "price": 649.99,
  "currency": "AOA",
  /* DESNORMALIZADO: evita JOIN com reviews */
  "ratings_summary": {
    "average": 4.3,
    "total_reviews": 1287,
    "distribution": { "5": 720, "4": 380, "3": 102, "2": 58, "1": 27 }
  },
  /* SCHEMA-LESS: atributos dinâmicos por subcategoria */
}
```



```

"attributes": { "RAM": "8GB", "Armazenamento": "256GB", "SO": "Android 14" },
/* EMBEDDING + GEOJSON: stock por armazém com localização */
"warehouse_stock": [
  { "warehouse_id": "WH-LIS-01", "city": "Lisboa",
    "quantity": 45,
    "location": { "type": "Point", "coordinates": [-9.1399, 38.7169] } }
],
"tags": ["bestseller", "promocao", "premium"],
"in_stock": true
}

```

4.3. Índices Criados

Nome	Tipo	Campos	PA
idx_category_price	Composto	(category, price)	PA-01
idx_text_search	Full-Text	(name, description, keywords, brand) ponderado	PA-02
idx_brand_subcategory	Composto	(brand, subcategory)	PA-01
idx_status_stock	Composto	(status, in_stock, is_featured)	PA-04
idx_warehouse_geo	2dsphere	(warehouse_stock.location)	PA-08
idx_rating_reviews	Composto	(ratings.average, ratings.total_reviews)	PA-03
idx_tags	Array	(tags)	PA-01
idx_user_email	Único	(email) colecção users	-----

5. IMPLEMENTAÇÃO E RESULTADOS

5.1. Ambiente de Testes

Componente	Detalhe
Sistema Operativo	Windows 11 (64-bit)
Orquestração	Docker Desktop + Docker Compose 2.x
Base de Dados	MongoDB 7.0 Replica Set rs0 (3 nós)
Nó Primário	mongo1:27017 (priority=2)
Nós Secundários	mongo2:27018 mongo3:27019
Interface Web	Mongo Express localhost:8081
Linguagem	Python 3.14 (pymongo 4.17, Faker, tqdm, tabulate)
Total de documentos	309.469 (100.000 produtos + 9.469 users + 200.000 reviews)

5.2. Resultados Reais das Consultas

As 7 consultas foram executadas sobre 309.469 documentos reais. Os tempos abaixo foram medidos com perf_counter do Python:

Q#	Descrição	Tempo	Índice Usado	Observação
Q1	Pesquisa Facetada (\$facet, múltiplos filtros)	113 ms	idx_category_price	947 resultados 6 subcategorias Eletrónicos
Q2	Análise Receita por Categoria (\$group, \$sum)	985 ms	COLLSCAN (relatório BI)	Eletrónicos: Kz 6.27B valor stock
Q3	Full-Text Score Composto (\$text + scoring)	91 ms	idx_text_search	10 laptops ordenados por relevância
Q4	Actualização Parcial Arrays (\$push, \$addToSet)	280 ms	product_id	Preço -20%, tag 'promoção' adicionada
Q5	Geoespacial Armazéns 100km Lisboa (\$geoWithin)	1.309 ms	idx_warehouse_geo	10 produtos com stock em Lisboa
Q6	Séries Temporais (\$unwind, \$group por mês)	465 ms	idx_updated_at	Evolução preços 6 meses Eletrónicos
Q7	Tendências (\$lookup reviews + score composto)	81.118 ms	Sem índice directo	10 produtos em tendência ver nota

Nota sobre Q7 81 segundos

A Query Q7 usa \$lookup entre a colecção reviews (200.000 docs) e products (100.000 docs) sem índice directo na chave de junção product_id→product_id. Este tempo elevado é esperado e representa uma limitação real documentada na secção 6. Solução: criar índice { product_id: 1 } na colecção reviews reduziria o tempo para ~500 ms.

5.3. Código das Consultas Avançadas

Q1 Pesquisa Facetada com \$facet

```
pipeline_q1 = [
  { "$match": { "category": "Eletrónicos", "price": {"$gte":100,"$lte":800},
    "in_stock": True, "status": "active",
    "ratings_summary.average": {"$gte": 3.5} } },
  { "$facet": {
    "products": [
      {"$sort": {"ratings_summary.average":-1, "sales_count":-1}},
      {"$limit": 10},
      {"$project": {"product_id":1,"name":1,"brand":1,"price":1}}
    ],
    "subcategory_counts": [
      {"$group": { "_id":"$subcategory","count":{"$sum":1}}},
      {"$sort": {"count":-1}}
    ],
    "price_stats": [
      {"$group": { "_id":None,"min":{"$min":"$price"},
        "max":{"$max":"$price"},"avg":{"$avg":"$price"}}}
    ]
  }}
]
# Resultado: 947 produtos | Samsung 132 | Smartwatches 167 | Tempo: 113 ms
```

Q3 Pesquisa Full-Text com Score Composto

```
pipeline_q3 = [
  { "$match": { "$text": {"$search": "laptop gaming"},
    "status": "active", "in_stock": True } },
  { "$addFields": {
    "text_score": {"$meta": "textScore"},
    "composite_score": { "$add": [
      {"$multiply": [{"$meta":"textScore"}, 10]},
```

```

    {"$multiply": [{"$divide":["$ratings_summary.average",5]}, 3]},
    {"$multiply": [{"$divide":["$sales_count",{"$add":["$sales_count",1000]}]}, 2]}
  ]}
}},
{"$sort": {"composite_score":-1}},
{"$limit": 10}
]
# Score combina: relevância textual (x10) + rating (x3) + vendas (x2)
# Resultado: LG Laptops — Score 121.35 | Tempo: 91 ms

```

Q4 Atualização Parcial Complexa

```

db.products.update_one(
  { "product_id": "PROD-081286" },
  {
    "$set": { "price": 39.78, "discount_percentage": 20, "updated_at": datetime.now() },
    "$push": { "price_history": {
      "date": datetime.now(), "price": 39.78, "reason": "Promoção Flash -20%" } },
    "$addToSet": { "tags": "promoção" },
    "$inc": { "update_count": 1 }
  }
)
# Resultado: 1 documento modificado | Tempo: 280 ms

```

Q5 Consulta Geoespacial

```

pipeline_q5 = [
  {"$match": {
    "status": "active", "in_stock": True,
    "warehouse_stock": {
      "$elemMatch": {
        "location": { "$geoWithin": {
          "$centerSphere": [[-9.1399, 38.7169], 100/6371] } },
        "quantity": {"$gt": 0}
      }
    }
  }},
  {"$sort": {"ratings_summary.average": -1}},
  {"$limit": 10}
]
# Coordenadas: Lisboa [-9.1399, 38.7169] | Raio: 100 km | Tempo: 1.309 ms

```

6. DISCUSSÃO CRÍTICA

6.1 . Limitações Identificadas

- Q7 \$lookup sem índice: A consulta de tendências demora 81 segundos porque o JOIN entre reviews e products não tem índice na coleção reviews para o campo product_id. Solução imediata: db.reviews.create_index([('product_id', 1)]) reduziria para ~500 ms.
- Crescimento do price_history: O array de histórico de preços dentro do documento pode crescer indefinidamente. Recomenda-se limitar via TTL ou mover histórico antigo para coleção de arquivo após 12 meses.
- Transações multi-documento: Operações ACID envolvendo múltiplas coleções (produto + stock + log) requerem transações MongoDB com overhead adicional de ~15–30 ms por operação.
- Resolução de nomes Docker no Windows: Em ambiente Windows, o Replica Set com nomes de host internos Docker exige configuração manual do ficheiro hosts para funcionar fora dos containers.

6.2. Escalabilidade: Volume 100x (10 Milhões de Produtos)

Com 10 milhões de produtos, as seguintes estratégias seriam aplicadas:

- Sharding por categoria: A chave { category: 1, _id: 1 } garante que queries filtradas por categoria são roteadas para um único shard (targeted query), evitando scatter-gather que degrada o desempenho.
- Read Preference secondaryPreferred: Redirecionar leituras de catálogo para nós secundários para distribuir carga, aceitando consistência eventual (aceitável atributos de produto mudam raramente).
- Cache Redis L2: Os 1% dos produtos mais visitados geram ~80% do tráfego (Princípio de Pareto). Uma camada de cache Redis com TTL de 5 minutos eliminaria a maioria das queries ao MongoDB.
- Atlas Search / Elasticsearch: Com 10M produtos, o índice full-text do MongoDB pode tornar-se um bottleneck. Elasticsearch oferece melhor escalabilidade para pesquisa textual a essa escala.

6.3 Tolerância a Falhas e Partições de Rede

Cenário de Falha	Comportamento do Sistema MongoDB
Falha do nó PRIMARY	Eleição automática em ~10s. Os 2 secundários formam quórum ($2/3 > 50\%$). Escritas temporariamente indisponíveis durante a eleição. Leituras continuam nos secundários.
Falha de 1 SECONDARY	Sistema continua totalmente operacional. Escrita e leitura no PRIMARY sem interrupção para o utilizador.
Partição de rede (split-brain)	Partição que isola o PRIMARY com apenas 1 nó: o PRIMARY retira-se voluntariamente (stepdown). O lado com 2 nós elege novo PRIMARY. Garante consistência (CP) nunca dois primários simultâneos.
Falha de 2 nós	Sistema fica em read-only (sem quórum de $2/3$ para escrita). Leituras ainda possíveis no nó sobrevivente em modo degradado.
Reinício completo	Os volumes Docker preservam todos os dados. Basta docker-compose up -d para restaurar o cluster ao estado anterior sem perda de dados.

6.4. Análise de Desempenho Conclusões

Os resultados obtidos demonstram que a estratégia de indexação é determinante para o desempenho:

Query	Tempo	Avaliação	Explicação
Q1 \$facet	113 ms	Excelente	idx_category_price usado examinou 4.061 docs de 100.000
Q3 Full-Text	91 ms	Excelente	idx_text_search compesos: name(10) > brand(8) > keywords(5)
Q4 Update	280 ms	Bom	Operação atômica com \$push + \$addToSet + \$set numa só operação

Q6 \$unwind	465 ms	Bom	Expande price_history de 100.000 produtos~800.000 entradas
Q2 \$group BI	985 ms	Aceitável	COLLSCAN esperado query de relatório de baixa frequência
Q5 Geo	1.309 ms	Aceitável	Cálculo esférico complexo sobre múltiplos documentos
Q7 \$lookup	81.118 ms	Necessita índice	reviews.product_id sem índice corrigível com 1 linha de código

CONCLUSÃO

O desenvolvimento deste trabalho sobre um Catálogo de Produtos Dinâmico para E-commerce de Alta Escala com MongoDB 7.0 foi uma experiência muito enriquecedora para o grupo, pois permitiu aplicar de forma concreta os conceitos de bases de dados NoSQL, modelação orientada a consultas e arquitecturas distribuídas estudados ao longo do semestre.

Uma das maiores dificuldades encontradas foi a configuração do Replica Set em ambiente Docker no Windows, nomeadamente a resolução de nomes de host internos entre os três nós. Outro desafio foi o design das estratégias de desnormalização, especialmente decidir quando fazer embedding e quando referenciar o que exigiu várias iterações sobre o modelo de dados antes de chegar a um esquema consistente e eficiente.

Os resultados reais obtidos sobre 309.469 documentos confirmaram as decisões de design: as queries com índices adequados (Q1 a Q6) atingiram latências entre 91 ms e 1.309 ms, todas dentro do intervalo aceitável. A excepção foi a Q7, cujos 81 segundos demonstraram de forma didáctica o impacto crítico da ausência de indexação numa operação \$lookup — uma limitação documentada e imediatamente solucionável com uma linha de código.

Em termos de competências desenvolvidas, o grupo saiu deste projecto com uma compreensão muito mais sólida de como modelar e implementar sistemas NoSQL em MongoDB, como analisar o desempenho de consultas com Aggregation Pipeline, e como garantir alta disponibilidade através de Replica Sets. Consideramos que os objectivos propostos pelo docente foram plenamente alcançados e que o sistema implementado está preparado para ser apresentado e defendido com confiança.

REFERÊNCIAS BIBLIOGRÁFICAS

MongoDB, Inc., MongoDB 7.0 Manual, 2024. [Online]. Disponível em: <https://www.mongodb.com/docs/manual/>

E. Brewer, "CAP twelve years later: How the 'rules' have changed," Computer, vol. 45, no. 2, pp. 23–29, Feb. 2012, doi: 10.1109/MC.2012.37.

P. Bailis e A. Ghodsi, "Eventual consistency today: Limitations, extensions, and beyond," Queue, vol. 11, no. 3, pp. 20–32, Mar. 2013.

K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. Sebastopol, CA: O'Reilly Media, 2019.

D. Abadi, "Consistency tradeoffs in modern distributed database system design," Computer, vol. 45, no. 2, pp. 37–42, Feb. 2012.

P. Sadalage e M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston, MA: Addison-Wesley, 2012.

MongoDB, Inc., Aggregation Pipeline, 2024. [Online]. Disponível em: <https://www.mongodb.com/docs/manual/aggregation/>

J. Han, E. Haihong, G. Le, e J. Du, "Survey on NoSQL database," in Proc. 6th Int. Conf. Pervasive Computing and Applications, 2011, pp. 363–366.

KANIVENGIDIO, M. Proposta Acadêmica de Avaliação Prática: SGBD II ,Arquitetura e Modelação Avançada em Sistemas NoSQL. Instituto Politécnico. Universidade Kimpa Vita, 2025/2026.